Table 1: *Excess* correlation ratio η (Roche et al., 1998) along axes of $\boldsymbol{V}$ (excess: subtract $d^{-0.5}$, so uniform random data $= 0.0$). This demonstrates substantial auto-correlation along the sequence axis. Calculated for Llama 2 7B over 40 SQuAD examples.

	B	S	Layer	Head	d_h
$\eta - d^{-0.5}$	0.143	**0.256**	0.0	0.0	0.0

Mean value reallocation In order to further improve the approximation in Equation (5), we note a further observation: $\boldsymbol{v}_i$ vectors within the sequence exhibit a high degree of auto-correlation (see Table 1). Thus, an additional correction term using a running-mean value vector $\bar{\boldsymbol{v}} = \frac{1}{S}\sum_{i=1}^{S} \boldsymbol{v}_i$ can be added as follows:

$$\boldsymbol{y}_2 = (\boldsymbol{s} \circ \boldsymbol{m}_{\boldsymbol{s}}) \cdot \boldsymbol{V} + (1 - \boldsymbol{s} \cdot \boldsymbol{m}_{\boldsymbol{s}})\bar{\boldsymbol{v}} \tag{6}$$

This introduces a minimal additional overhead compared to Equation (5) due to the mean vector $\bar{\boldsymbol{v}}$ being updated and written back to memory at each step.

Query sparsity In order to improve the lower bound on memory transfers, we further consider efficiently approximating the mask $\boldsymbol{m}_{\boldsymbol{s}}$ by calculating approximate attention scores $\hat{\boldsymbol{s}}$ without using the full matrix $\boldsymbol{K}$. Here, we consider the distribution of magnitudes of the components of the query vector $\boldsymbol{q}$ and observe that it is highly *heavy-tailed* (see Figures 4c and 4d). This observation allows us to efficiently approximate the attention scores $\boldsymbol{s}$ by defining a per-query boolean mask $\boldsymbol{m}_{\boldsymbol{q}} \in \{0,1\}^{d_h}$ corresponding to the top-r components of $\boldsymbol{q}$. The scores are then approximated as:

$$\hat{\boldsymbol{s}} = \text{softmax}\left(\frac{(\boldsymbol{q} \circ \boldsymbol{m}_{\boldsymbol{q}}) \cdot \boldsymbol{K}^\top}{\tau}\right) \tag{7}$$

where τ is the softmax temperature. Due to the mask $\boldsymbol{m}_{\boldsymbol{q}}$, only the components of $\boldsymbol{K}$ corresponding to non-zero elements of the mask need to be fetched from memory. The top-k mask $\boldsymbol{m}_{\hat{\boldsymbol{s}}} \in \{0,1\}^S$ can then be calculated using $\hat{\boldsymbol{s}}$ (see Figure 4e) and the approximate attention output is obtained as:

$$\boldsymbol{y}_3 = \text{softmax}\left(\frac{\boldsymbol{q} \cdot \boldsymbol{K}^\top}{\sqrt{d_h}} + \log(\boldsymbol{m}_{\hat{\boldsymbol{s}}} + \epsilon)\right) \cdot \boldsymbol{V} \tag{8}$$

with $\epsilon \to 0$. Again, due to the mask $\boldsymbol{m}_{\hat{\boldsymbol{s}}}$, only the key-value pairs corresponding to the non-masked elements need to be fetched from the memory.

Mean value reallocation with query sparsity As a final consideration, we look at combining the mean value reallocation improvement of Equation (6) with the approach in Equation (8). As we do not have access to the full scores $\boldsymbol{s}$, we proceed to approximate the weighted sum using the approximate scores in Equation (7). Note that, since the query-key dot product is performed over only r dimensions, care needs to be taken when choosing the appropriate softmax temperature τ in Equation (7). If r components were chosen randomly, the appropriate temperature would be $\sqrt{r}$. On the other hand, if the top-r components were the only non-zero elements of the query vector, the appropriate temperature would remain $\sqrt{d_h}$. As a balance between the two extremes, we have found the following temperature to yield a good approximation (see Figure 4f):

$$\tau = \sqrt{d_h \frac{\|\boldsymbol{q} \circ \boldsymbol{m}_{\boldsymbol{q}}\|_1}{\|\boldsymbol{q}\|_1}} \tag{9}$$

The final attention output can be then calculated as a weighted sum:

$$\boldsymbol{y} = \alpha\, \boldsymbol{y}_3 + (1 - \alpha)\bar{\boldsymbol{v}} \tag{10}$$

where $\alpha = \boldsymbol{m}_{\hat{\boldsymbol{s}}} \cdot \hat{\boldsymbol{s}}$ is the relative weight of the top-k terms.

4. SparQ Attention

Following the analysis in Section 3, we propose **SparQ Attention** (see Figure 5) consisting of three steps:

Step 1: Find the indices of r largest components of $|\boldsymbol{q}|$[1] and only fetch $\boldsymbol{K}$ along the dimensions corresponding to these indices. Calculate *approximate* attention scores $\hat{\boldsymbol{s}}$ using the sliced query and keys.

Step 2: Find the top-k positions in the approximate attention scores and fetch the corresponding *full* key and value vectors. Calculate the output of the attention operation using the top-k keys and values.

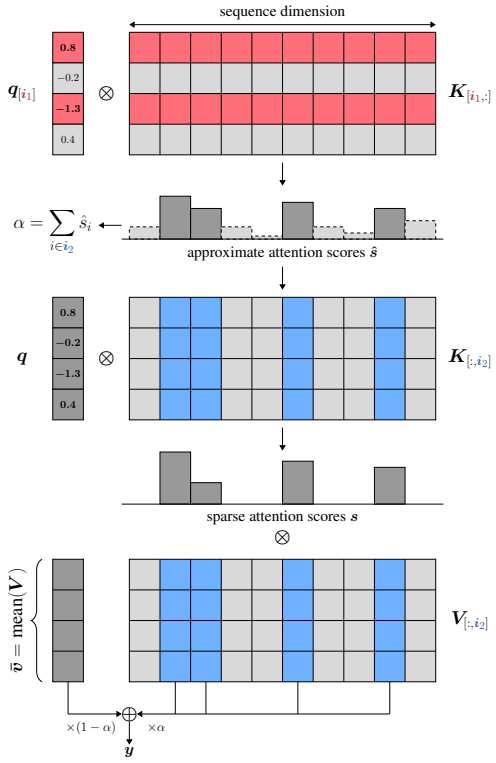
Step 3: Estimate the total score α assigned to the top-k positions using the approximate attention scores. Use this total score to interpolate between the attention output from the top-k positions, and a *mean value* vector, $\bar{\boldsymbol{v}}$.

The memory transfer of the SparQ Attention algorithm for a single attention head forward-pass:

$$\mathcal{M}_{\text{SparQ}} = S\,r \; + 2\,k\,d_h + 4\,d_h \tag{11}$$

where the first term corresponds to reading r rows of $\boldsymbol{K}$, the second term corresponds to reading the top-k columns of $\boldsymbol{K}$ and $\boldsymbol{V}$ and the third term corresponds to transfers associated with writing the current $\boldsymbol{k}$ and $\boldsymbol{v}$, in addition to reading and writing $\bar{\boldsymbol{v}}$.

[1] We use $|\cdot|$ to denote element-wise absolute value.



Algorithm 1 SparQ Attention

Input: $\boldsymbol{q} \in \mathbb{R}^{d_h}$, $\boldsymbol{K} \in \mathbb{R}^{S \times d_h}$, $\boldsymbol{V} \in \mathbb{R}^{S \times d_h}$, $\overline{\boldsymbol{v}} \in \mathbb{R}^{d_h}$, $r \in \mathbb{N}$, $k \in \mathbb{N}$, $l \in \mathbb{N}$

 # Indices of top r elements of $|\boldsymbol{q}|$
$\boldsymbol{i}_1 \leftarrow \text{argtopk}\left(|\boldsymbol{q}|, r\right)$

 # Softmax temperature, weighted by L1 coverage
$\tau \leftarrow \sqrt{d_h \cdot \frac{\|\boldsymbol{q}_{[\boldsymbol{i}_1]}\|_1}{\|\boldsymbol{q}\|_1}}$

 # Approximate attention scores (all positions)
$\hat{\boldsymbol{s}} \leftarrow \text{softmax}\left(\boldsymbol{q}_{[\boldsymbol{i}_1]} \cdot \boldsymbol{K}^\top_{[\boldsymbol{i}_1,:]} / \tau\right)$

 # Local mask of last l positions
$\boldsymbol{m} \leftarrow [1 \textbf{ if } i > S - l \textbf{ else } 0]_{i=1}^{S}$

 # Indices of top k approximate scores or local
$\boldsymbol{i}_2 \leftarrow \text{argtopk}\left(\hat{\boldsymbol{s}} + \boldsymbol{m}, k\right)$

 # Total approximate score of top k
$\alpha \leftarrow \text{sum}\left(\hat{\boldsymbol{s}}_{[\boldsymbol{i}_2]}\right)$

 # Final attention scores (top k positions)
$\boldsymbol{s} \leftarrow \text{softmax}\left(\boldsymbol{q} \cdot \boldsymbol{K}^\top_{[:,\boldsymbol{i}_2]} / \sqrt{d_h}\right)$

 # Mixed scores and values, interpolating with $\overline{\boldsymbol{v}}$
$\boldsymbol{y} \leftarrow \alpha\, \boldsymbol{s} \cdot \boldsymbol{V}_{[:,\boldsymbol{i}_2]} + (1 - \alpha)\, \overline{\boldsymbol{v}}$

return $\boldsymbol{y}$

Figure 5: SparQ Attention for a single attention head. The algorithm consists of three steps. First, we find the r largest components of the incoming query vector and gather the corresponding components along the hidden dimension of the key cache $\boldsymbol{K}$. This allows us to approximate the full attention scores ($\hat{\boldsymbol{s}}$). In the second step, we identify the top-k largest scores in the approximation and proceed to gather the corresponding full key and value vectors from the cache. As a final step, to compensate for the missing value vectors, we additionally maintain and fetch the running mean value vector $\overline{\boldsymbol{v}}$ and reassign it the leftover mass based on approximate score weightings. The attention output is then calculated as usual using the top-k fetched key and value pairs, together with $\overline{\boldsymbol{v}}$.

By varying r and k, we can tune the total amount of data transferred by the scheme, trading-off approximation accuracy for token-generation speed-up. Since typically $S \gg d_h$, r is the most important parameter controlling the data transfer compression ratio $\mathcal{M}_{\text{SparQ}}/\mathcal{M}_{\text{dense}}$.

Grouped query attention For models using GQA, groups of g queries access the same KV head. In order to accommodate this, we modify **Step 1** to sum $|\boldsymbol{q}|$ within each group before selecting top-r components. Similarly, **Step 2** is modified by summing the approximate attention scores within each group before selecting top-k keys and values for each KV head. Although **Step 3** can be implemented exactly as before, we found that GQA models obtained better performance without it, so we omitted this step for Llama 3 and Mistral. The full code can be found in Appendix B.

5. Experiments

5.1. Setup

Models We evaluate our method on five widely-used open-source language model variants: Llama 2 (Touvron et al., 2023), Llama 3 (Meta AI, 2024), Mistral (Jiang et al., 2023), Gemma (Mesnard et al., 2024) and Pythia (Biderman et al., 2023), evaluating model sizes up to 13 billion parameters.[2] All models are decoder-only transformers (Radford et al., 2018), pre-trained on causal language modelling. They share similar architectural components such as rotary positional embedding (Su et al., 2021), while also having some notable differences such as different attention mechanisms (multi-head and grouped query attention), layer normalisation implementations, activation functions and execution of modules in parallel.

Tasks In order to evaluate our method on a spectrum of relevant NLP tasks that present a particular challenge to sparse attention techniques, our evaluation setup consists of various tasks requiring information retrieval and reasoning over long input sequences. This includes question answering, summarisation, perplexity/bits-per-character (BPC), and text repetition. For this, we adapted standard downstream tasks and datasets to generate examples of sequence lengths be-

[2] https://github.com/graphcore-research/llm-inference-research/tree/2024-05-sparq

Table 2: Results for the largest model of each family tested are presented below. SQuAD and TriviaQA measure performance in accuracy as a percentage; CNN/DailyMail uses ROUGE-L score; WikiText task measures perplexity in bits per character (BPC); Repetition counts the number of characters before the generation diverges. Values presented are those closest to the target compression ratio for each technique, where bold represents the best score for each setting. Median standard errors across all models and sparsity settings are: SQuAD 0.8, TriviaQA 0.8, CNN/DailyMail 0.4, WikiText 0.01, Repetition 2.

Dataset Name		SQuAD ↑			TriviaQA ↑			CNN/DailyMail ↑			WikiText ↓			Repetition ↑		
Compression		1	1/2	1/8	1	1/2	1/8	1	1/2	1/8	1	1/2	1/8	1	1/2	1/8
Llama 2 13B	LM-∞	**80.8**	50.0	30.1	**78.7**	73.4	68.1	**22.1**	16.8	14.9	**0.61**	0.64	0.71	**229**	76	29
	H_2O	**80.8**	73.2	63.0	**78.7**	78.5	**78.4**	**22.1**	22.2	20.3	**0.61**	**0.61**	**0.64**	**229**	61	26
	SparQ	**80.8**	**80.7**	**74.9**	**78.7**	**78.8**	78.2	**22.1**	**22.5**	**21.6**	**0.61**	**0.61**	0.70	**229**	**227**	**190**
Llama 3 8B	LM-∞	**81.2**	66.0	51.8	**83.2**	81.8	80.6	**23.4**	17.1	16.1	**0.56**	0.58	0.64	**213**	102	27
	H_2O	**81.2**	74.5	61.7	**83.2**	**83.2**	82.5	**23.4**	23.3	21.9	**0.56**	**0.56**	0.59	**213**	67	30
	SparQ	**81.2**	**81.2**	**78.3**	**83.2**	83.0	**82.8**	**23.4**	**23.4**	**23.4**	**0.56**	0.57	**0.58**	**213**	**214**	**213**
Mistral 7B	LM-∞	**81.0**	51.0	29.0	**80.9**	75.8	72.6	**23.7**	18.0	16.6	**0.62**	0.65	0.72	**231**	81	20
	H_2O	**81.0**	71.2	56.9	**80.9**	**80.8**	**80.2**	**23.7**	**23.5**	22.8	**0.62**	**0.63**	0.66	**231**	38	14
	SparQ	**81.0**	**80.9**	**77.5**	**80.9**	**80.8**	79.0	**23.7**	**23.5**	**23.0**	**0.62**	**0.63**	**0.65**	**231**	**209**	**201**
Gemma 7B	LM-∞	**80.4**	64.2	48.7	**82.8**	81.6	80.8	**17.4**	13.1	13.3	**0.59**	0.61	0.68	**245**	101	23
	H_2O	**80.4**	73.7	60.2	**82.8**	**82.9**	82.5	**17.4**	17.4	16.9	**0.59**	**0.59**	0.62	**245**	68	18
	SparQ	**80.4**	**80.3**	**80.3**	**82.8**	82.8	**82.7**	**17.4**	**17.9**	**18.0**	**0.59**	**0.59**	**0.59**	**245**	**240**	**237**
Pythia 6.9B	LM-∞	**57.8**	38.5	17.0	**52.6**	41.6	29.7	**20.2**	14.9	14.0	**0.68**	0.71	0.79	**150**	64	18
	H_2O	**57.8**	52.9	45.5	**52.6**	**52.6**	**52.3**	**20.2**	20.3	18.5	**0.68**	0.69	0.71	**150**	47	17
	SparQ	**57.8**	**58.0**	**57.1**	**52.6**	52.4	51.7	**20.2**	**20.6**	**20.6**	**0.68**	**0.68**	**0.70**	**150**	**151**	**144**

tween 1k and 2k tokens. To define the tasks independently of the selected models, our examples were chosen to have sequence lengths between 4000 and 8000 characters, roughly giving the desired lengths in tokens.

For question answering, we use the SQuAD (Rajpurkar et al., 2016) and TriviaQA (Joshi et al., 2017) datasets in the *open-book* setting. In order to construct the SQuAD examples, we augment the provided context (i.e. the standard SQuAD input sequence required to answer the question) with seven additional "*confusion contexts*" from unrelated questions. This ensures that the examples have a large sequence length, while making the task harder as the model needs to distinguish the relevant information from the context from the unrelated paragraphs. We use SQuAD v1.1, as it does not include unanswerable questions included in SQuAD v2.0, since we aim to measure the model's ability to extract useful information from the KV cache. For both question answering tasks we use exact string match accuracy as the evaluation metric. Summarisation is evaluated on the CNN/DailyMail dataset (See et al., 2017) using the ROUGE-L F-score (Lin, 2004) as the metric. We use the WikiText-103 dataset (Merity et al., 2016) with bits per character (BPC) for evaluating language modelling performance.[3] Finally, we construct an artificial "Text Repetition" task to evaluate the capability of the model to repeat sentences from its context verbatim. Such a task can commonly appear in a dialogue setting where the LLM agent is required to retrieve a piece of text from a possibly long context provided, and can be challenging for sparse attention techniques. We construct examples using the Tiny-Shakespeare dataset (Karpathy, 2015) by chunking the text into contexts of the appropriate size, appending them with the prompts containing a subset of the context, and evaluating the output exact character length match with the continuation from the context.

[3]We quote performance for sub-word language modelling in BPC, to account for any differences in vocabulary across models.

Baselines We consider the cache eviction technique H_2O (Zhang et al., 2023), top-k sparse attention in the form of FlexGen (Sheng et al., 2023), and LM-Infinite, a local windowing scheme with initial-tokens included proposed by Han et al. (2023) as baselines. For each experiment we fix the KV cache transfer budget k idependently of the sequence length. With H_2O, we set the local window size $l = k/4$ (with $3k/4$ heavy hitters), and for LM-Infinite we always include the first 16 positions (with $k - 16$ local positions). Due to the lower bound of FlexGen's compression ratio being $1/2$, we do not report the technique's results in Table 2 and Table 3, but full results can be found in Appendix A. The compression ratio definitions for each of these techniques can be found in Appendix G.

5.2. Results

Our experiments span eight distinct models: Llama 2 with 7 and 13 billion parameters, Llama 3 with 8 billion parameters, Mistral with 7 billion parameters, Gemma with 7 billion parameters, and three Pythia models with 1.4, 2.8 and 6.9